

배포



1. Supabase Database 설정

❖ Supabase에서 Project 생성

- Supabase 대시보드 화면 (<https://supabase.com/dashboard>)에서 미리 생성한 Org로 이동
 - 2장 환경 설정에서 이미 조직 단위를 생성했었음
- 새 프로젝트 생성
 - Project Name : 자신만의 프로젝트명 (예: MyProject)
 - Database Password : 자신만의 패스워드 설정
 - Region : Northeast Asia (Seoul) 선택
- 데이터베이스 연결 문자열 확인
 - 프로젝트 화면에서 상단의 connect 버튼 클릭
 - Transaction Pooler의 Connection String 복사 후 백업
 - 예시 : postgresql://postgres.xkntxvgelibwtaivisly:[YOUR-PASSWORD]@aws-1-ap-northeast-2.pooler.supabase.com:6543/postgres
 - 이 값에 Password만 지정하면 됨

1. Supabase Database 설정

❖ 배포를 위한 mcp 설정

- .mcp.deploy.json

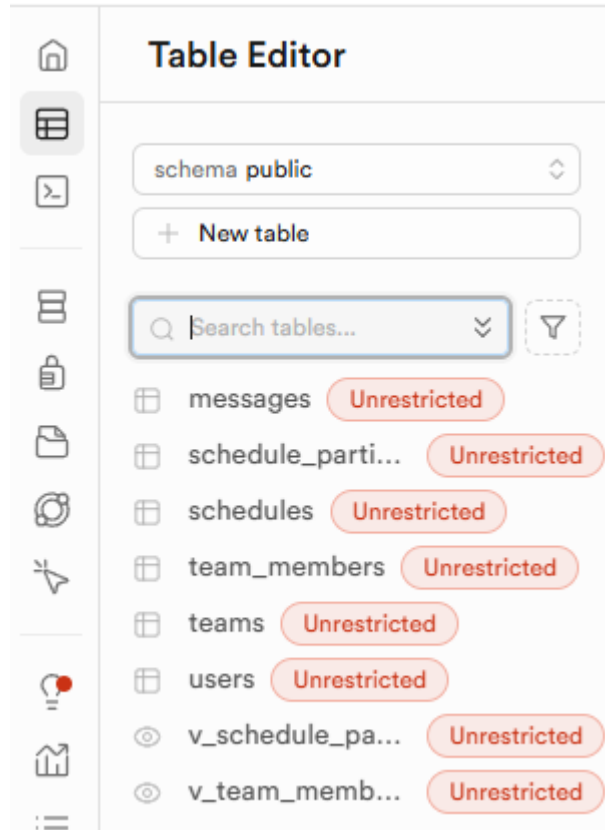
```
{
  "mcpServers": {
    "supabase": {
      "type": "http",
      "url": "https://mcp.supabase.com/mcp?features=docs,account,database,debugging,development,functions,branching"
    },
    "postgresql-mcp": {
      "command": "npx",
      "args": [ "-y", "@henkey/postgres-mcp-server" ]
    }
  }
}
```

- `claude --mcp-config .mcp.deploy.json` 실행
- `/mcp` 명령어로 연결 여부 확인하고 브라우저로 supabase 연결 시도

1. Supabase Database 설정

❖ 다음과 같은 프롬프트로 로컬 데이터베이스를 supabase로 마이그레이션

- "postgresql-mcp와 supabase mcp를 이용해 로컬 데이터베이스의 스키마를 supabase로 마이그레이션 해줘"
- 마이그레이션 후 supabase 브라우저 화면에서 생성된 테이블 확인



2. vercel 소개

❖vercel 이란?

- 프론트엔드 개발자를 위한 클라우드 플랫폼으로, 웹 애플리케이션을 쉽고 빠르게 배포하고 호스팅할 수 있게 해주는 서비스

❖vercel의 특징

- 복잡한 설정 없이 git 저장소와 연결만으로 손쉬운 배포
- 글로벌 CDN 제공
- HTTPS 기본적용 및 커스텀 도메인 지원

❖vercel에 배포할 수 있는 기술 스택

- 대부분의 프론트엔드 기술 : React, Next.js, Vue 등
- Serverless Functions : 서버리스 함수를 이용해 백엔드 API를 간단하게 구성할 수 있음
 - 가능한 배포 : Node.js, Python, Go, Ruby

2. vercel for github

❖vercel for github 이란?

- vercel에 내장된 github 연동 배포 기능
- Github에 코드를 푸시하면 자동으로 vercel에 배포함
- main 또는 master 브랜치에 코드가 병합되면 Production 환경으로 배포
- 다른 브랜치는 Preview 환경으로 배포됨

❖vercel for github의 장점

- 설정이 매우 간단함 : Vercel 대시보드 화면에서 몇번의 클릭만으로 배포 가능
- CI/CD 파이프라인을 별도로 구성할 필요없음

3. vercel for github을 이용한 배포

❖배포전 확인할 것

- 환경변수 : 특히 데이터베이스 연결 문자열

❖vercel 에서 백엔드 배포 예시

- vercel 대시보드 화면에서 오른쪽 상단의 'Add New' 클릭 - Project 선택
- Import Git Repository에서 깃헙 리포지토리 연결
 - 프로젝트명 : team-caltalk-backend
 - Root Directory : Edit 클릭 후 backend 선택
- 배포 후 환경변수 설정
 - 배포된 앱의 Settings - Environment Variables 클릭 후 환경 변수 입력
 - 입력할 값
 - Environments : 지정하지 않으면 'All Environments'
 - Key - Value
 - Production 환경과 Preview 환경이 다른 값을 사용할 경우 명시적으로 지정해주어야 함
- 배포된 앱이 정상 실행되지 않을 경우
 - Vercel의 로그를 바탕으로 프롬프팅하여 문제 해결

3. vercel for github을 이용한 배포

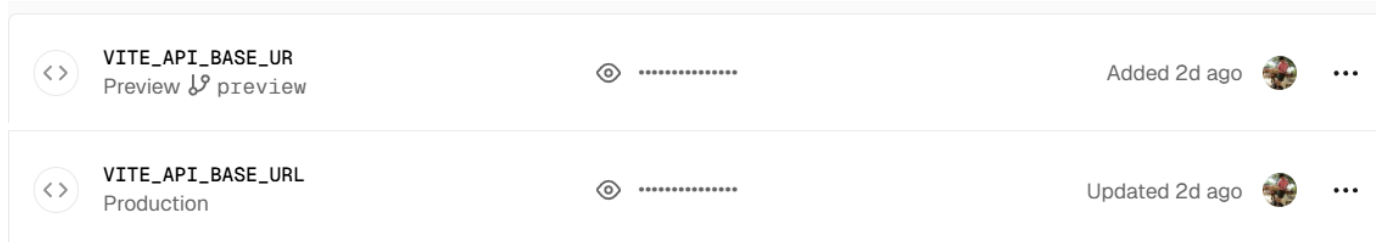
■ 백엔드 환경 변수 설정 예

<>	CORS_ORIGIN Production and Preview	👁 [REDACTED]	Updated 2d ago		...
<>	NODE_ENV Preview ↱ preview	👁 [REDACTED]	Added 2d ago		...
<>	NODE_ENV Production	👁 [REDACTED]	Added 2d ago		...
<>	DB_CONNECTION_STRING All Environments	👁 [REDACTED]	Added 2d ago		...
<>	JWT_SECRET All Environments	👁 [REDACTED]	Added 2d ago		...
<>	JWT_REFRESH_SECRET All Environments	👁 [REDACTED]	Added 2d ago		...
<>	JWT_REFRESH_EXPIRE All Environments	👁 [REDACTED]	Added 2d ago		...
<>	JWT_EXPIRE All Environments	👁 [REDACTED]	Added 2d ago		...

3. vercel for github을 이용한 배포

❖vercel 에서 프론트 배포 예시

- vercel 대시보드 화면에서 오른쪽 상단의 'Add New' 클릭 - Project 선택
- Import Git Repository에서 깃헙 리포지토리 연결
 - 프로젝트명 : team-caltalk-frontend
 - Root Directory : Edit 클릭 후 frontend 선택
- 배포 후 환경변수 설정
 - 백엔드에서와 동일한 방법으로 설정
- 프론트엔드 환경변수 설정 예



❖배포된 앱의 기본 주소

- Production : 프로젝트명.vercel.app
- Preview : 프로젝트명-고유해시.vercel.app

3. vercel for github을 이용한 배포

❖ 배포 후 애플리케이션 작동 여부 확인

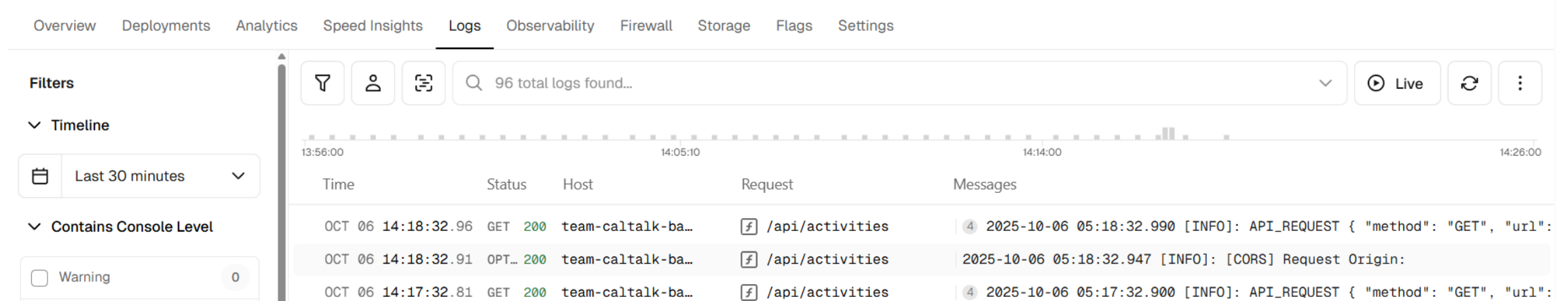
- 오작동한다면?
 - 환경변수 설정값 확인 -> 변경
 - 애플리케이션 재배포할 것

The screenshot shows the Vercel team dashboard for 'MyApp 개발 팀1'. At the top, there's a navigation bar with '팀캘린더', '대시보드', '팀', '캘린더', 'MyApp 개발 팀1', '원형설정', and '로그아웃'. The main header says '안녕하세요, 원형섭님!' with a greeting icon and the current time '현재 시각 14:17'. Below this, it says '오늘은 2025년 10월 06일 월요일이고, 현재 시각은 14:17입니다'. The main content area features a team card for 'MyApp 개발 팀1' with a '현재 활성 팀' status and a '완료' button. Below the team card are three sections: '팀 관리' (Team Management) with a '내 팀 목록' button and '+ 팀 생성' and '팀 참여' buttons; '캘린더' (Calendar) with a '캘린더 열기' button; and '최근 활동' (Recent Activity) showing a list of events like '팀장 일정11' and '변경1' with status indicators. At the bottom, there's a section for '내 팀 (2개)' (My Teams) showing 'MyApp 개발 팀2' and 'MyApp 개발 팀1' (selected).

3. vercel for github을 이용한 배포

❖vercel 백엔드 배포시 주의사항

- Postgresql DB 접속 라이브러리를 prisma를 사용했을 때 배포오류 발생
 - 다른 라이브러리를 사용하도록 코드를 변경할 것 : 예) pg 라이브러리
- 백엔드를 위한 Serverless Function 환경에서는 파일 시스템에 쓰기가 제한되어 있음
 - /tmp 디렉토리만 쓰기 가능
 - 로그를 파일시스템에 남기도록 했다면 배포 실패
- 백엔드 배포는 파일시스템에 로깅하는 코드를 Console Log로 코드를 변경해야 함
 - logger 함수를 만들어서 사용하고 있다면 logger만 변경하면 됨
 - Console Log 는 Vercel의 Log 화면에서 확인할 수 있음



The screenshot shows the Vercel Logs interface. At the top, there are tabs for Overview, Deployments, Analytics, Speed Insights, Logs (selected), Observability, Firewall, Storage, Flags, and Settings. Below the tabs, there's a 'Filters' section on the left with a 'Timeline' dropdown set to 'Last 30 minutes' and a 'Contains Console Level' section with a 'Warning' filter set to 0. The main area displays a search bar with '96 total logs found...' and a timeline view. Below the timeline, there's a table with columns: Time, Status, Host, Request, and Messages. The table shows three log entries for a GET request to /api/activities.

Time	Status	Host	Request	Messages
OCT 06 14:18:32.96	GET 200	team-caltalk-ba...	/api/activities	2025-10-06 05:18:32.990 [INFO]: API_REQUEST { "method": "GET", "url":
OCT 06 14:18:32.91	OPT... 200	team-caltalk-ba...	/api/activities	2025-10-06 05:18:32.947 [INFO]: [CORS] Request Origin:
OCT 06 14:17:32.81	GET 200	team-caltalk-ba...	/api/activities	2025-10-06 05:17:32.900 [INFO]: API_REQUEST { "method": "GET", "url":

4. CI/CD

❖CI/CD 개념

■ CI/CD란?

- 애플리케이션 개발부터 배포까지의 전체 과정을 자동화하여 코드 변경사항을 더 자주 빌드, 테스트하여 신뢰할 수 있는 방식으로 배포하는 배포 자동화 방법

■ CI(Continuous Integration) : 지속적 통합

- 개발자들이 새로운 코드 변경 사항이 리포지토리에 병합될 때 빌드, 테스트를 자동화하여 테스트되고 검증된 코드가 운영환경에 배포될 수 있도록 준비하는 작업
- Trunk(일반적으로 main 브랜치)에는 테스트되고 검증된 코드만 병합되도록 함

■ CD(Continuos Delivery) : 지속적 전달

- CI 과정을 모두 포함하며, CI 과정을 거친 코드를 릴리스 준비가 된 상태로 만듦
- 수동 승인을 거쳐서 Production 환경으로의 배포가 이루어짐

■ CD(Continuos Deployment) : 지속적 배포

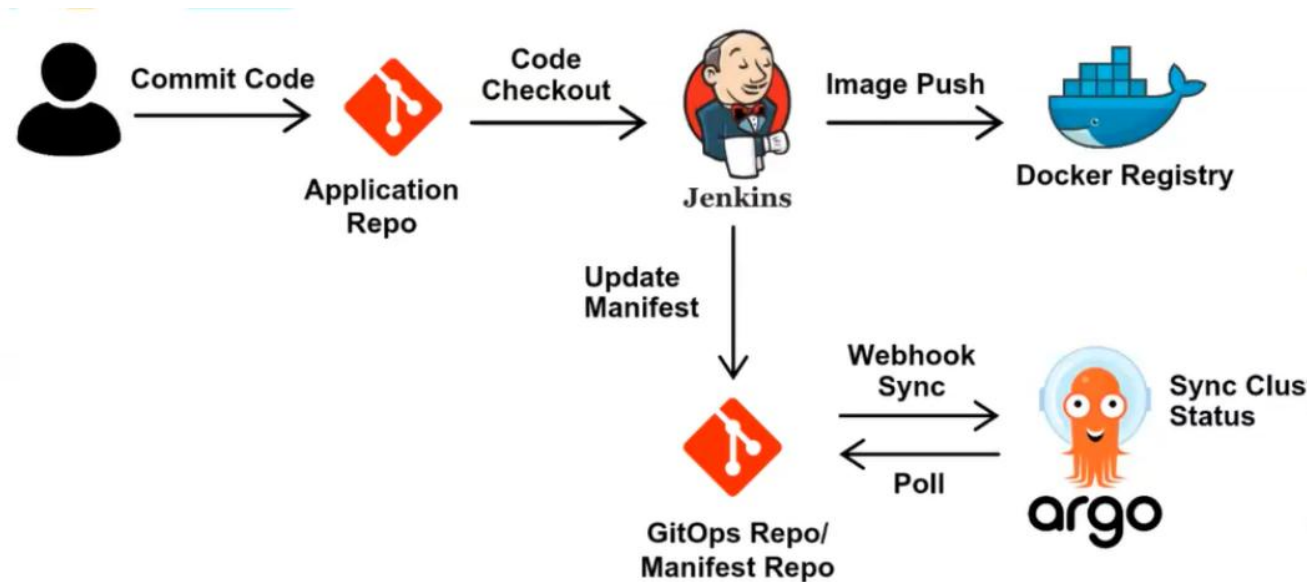
- 지속적인 전달이 확장된 개념
- 자동화된 테스트를 통과한 모든 변경 사항을 자동으로 Production 환경에 배포함.
- 수동 승인 과정 없음

4. CI/CD

❖ CI/CD 자동화 도구

- Jenkins
- Github Action : 이 과정에서는 github action만을 사용
- Gitlab CI

❖ Jenkins 사용 예시 : jenkins + k8s + argocd



참조 : <https://medium.com/@lilnya79/gitops-argocd-continuous-delivery-with-jenkins-and-github-8d19ac5ece4c>

4. github action

❖github action 이란?

- GitHub 저장소에서 코드 이벤트(Push, PR, 릴리즈 등)를 트리거로 자동화 작업(CI/CD, 반복 스크립트 실행)을 실행하게 해주는 기능

❖핵심 요소

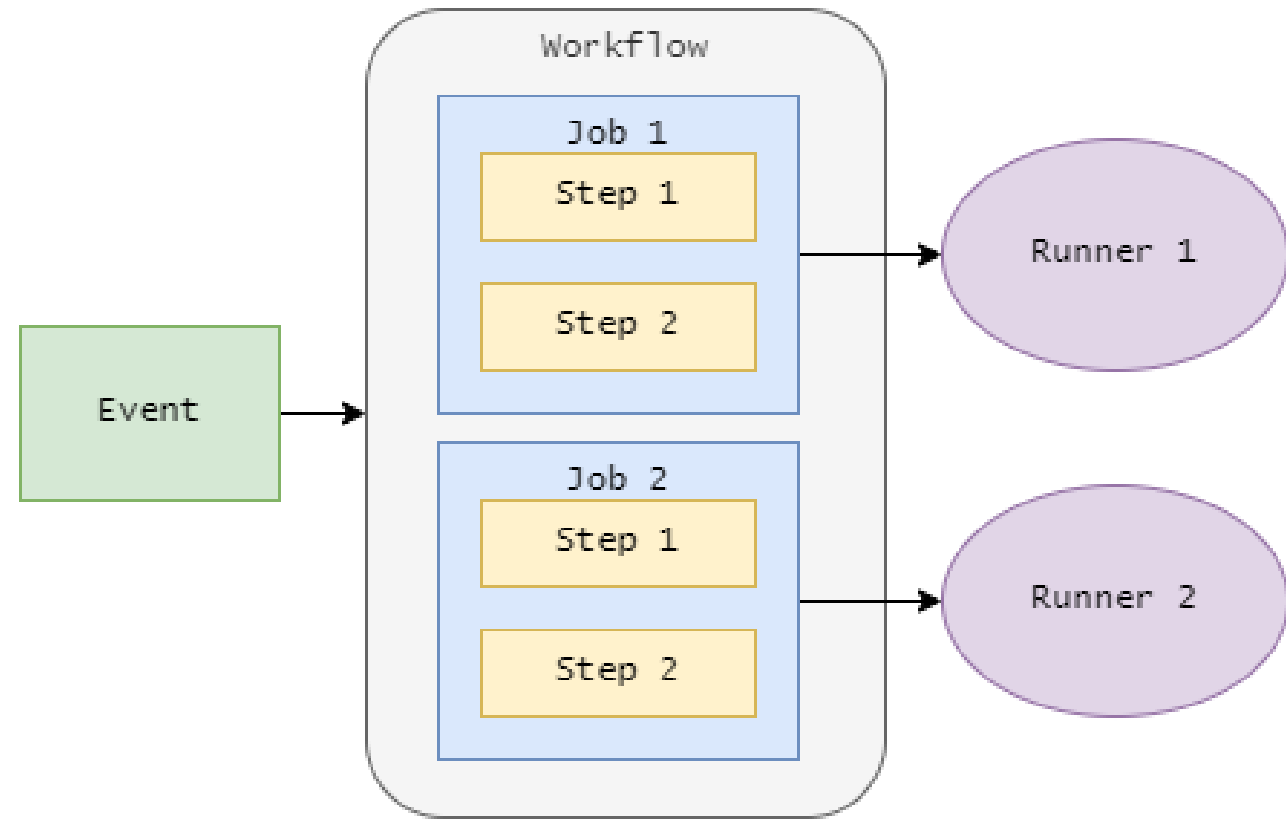
- Workflow
 - .github/workflows/ 디렉토리에 YAML 파일로 작성하는 자동화 스크립트
- Events
 - 특정 이벤트(push, PR, issue) 발생시에 Job을 실행하도록 하는 트리거
- Jobs
 - 병렬/순차로 실행되는 작업 단위. 각 Job은 하나의 러너에서 실행.
 - Job 내부에는 세부 단계를 지정하는 Shell Script또는 재사용 가능한 Action으로 구성된 Step을 배치
 - 재사용할 수 있는 액션 목록 : <https://github.com/actions>
- Runners
 - 작업이 실행되는 머신. GitHub이 제공하는 호스티드 러너 또는 Self-hosted.
 - 기본 제공되는 Runner : ubuntu, windows. macos

4. github action

❖github action 장점

- 별도의 서버, 시스템을 구성할 필요가 없음
 - github에 내장된 기능
- 다양한 Action 지원
 - github 지원
 - <https://github.com/actions>
 - 이밖에도 3rd-party가 제공하는 것들도 많음

❖github action의 아키텍처



4. github action

❖github workflow 예시 : .github/workflows/test-build

- preview 브랜치에 push 되면 테스트 후 빌드

```
name: build-test-example-workflow
# events
on:
  push:
    branches: [preview]
# Jobs
jobs:
  test:
    # Runner
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [22.x, 24.x]
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
- name: Setup Node.js ${{ matrix.node-version }}
  uses: actions/setup-node@v4
  with:
    node-version: ${{ matrix.node-version }}
    cache: "npm"

- name: Install dependencies
  run: npm ci

- name: Run tests
  run: npm run test

- name: Build
  run: npm run build
```


4. github action

❖ Vercel for github 과 Github Action을 이용한 배포 비교

- Vercel for github
 - Vercel이 Github과 직접 통합
 - Git 이벤트를 Vercel이 직접 감지하고 처리함
 - 설정이 간단하지만 배포 프로세스에 대한 커스터마이징이 어려움
 - Vercel 이외의 다른 타겟으로의 배포 불가능
- Github Action
 - 배포 작동 : Github 리포지토리 --> Github Actions --> Vercel
 - Github Action이 중간 레이어 역할 수행
 - 초기 설정이 복잡하지만 다양한 설명 가능
 - 테스트/린트/빌드 등의 작업을 직접 제어할 수 있음
 - 다양한 타겟으로 배포할 수 있음

5. github action을 이용한 vercel 배포로 전환

❖사전 작업

▪ Vercel 토큰, 조직 ID, Project ID 확인

• 토큰

– <https://vercel.com/account/settings/tokens> 에서 생성한 후 백업해둘 것

TOKEN NAME	SCOPE	EXPIRATION
team-caltalk-token	Full Account	No Expiration

• 조직 ID

– 대시보드 화면에서 Settings - General에서 Team ID를 백업해둘 것

• 프로젝트 ID

– 프론트엔드, 백엔드 각각 프로젝트 화면에서 Settings - General에서 Project ID를 복사해둘 것

▪ github action의 secret 설정

• github 리포지토리의 settings로 이동 후 왼쪽 패널에서 'Secrets and variables - Actions' 로 이동

• 4개의 Repository Secret 설정

– VERCEL_TOKEN

– VERCEL_ORG_ID

– VERCEL_PROJECT_ID_BACKEND, VERCEL_PROJECT_ID_FRONTEND

5. github action을 이용한 vercel 배포로 전환

- 기존 vercel for github의 배포 중단 설정
 - 프론트엔드와 백엔드 디렉토리에 vercel.json 파일을 생성하고 다음 설정을 추가

```
{
  .....(생략)
  "git": {
    "deploymentEnabled": false
  }
}
```

❖ 백엔드 배포를 위한 workflow 파일 예시

- backend-preview.yaml : preview 브랜치에 푸시하거나 main 브랜치에 푸시될때 실행

```
name: Backend Preview
on:
  push:
    branches: [preview]
    paths:
      - "backend/**"
  pull_request:
    branches: [main]
    paths:
      - "backend/**"
```

```
jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [22.x]
```

5. github action을 이용한 vercel 배포로 전환

❖백엔드 배포를 위한 workflow 파일 예시

▪ backend-preview.yaml(이어서)

```
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js ${{ matrix.node-version }}
    uses: actions/setup-node@v4
    with:
      node-version: ${{ matrix.node-version }}
      cache: "npm"

  - name: Install dependencies
    working-directory: ./backend
    run: npm ci

  - name: Run tests
    working-directory: ./backend
    run: npm run test
```

test job이 완료되어야 deploy 가 진행

```
deploy:
  needs: test
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Deploy to Vercel
      uses: amondnet/vercel-action@v25
      with:
        vercel-token: ${{ secrets.VERCEL_TOKEN }}
        vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
        vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID_BACKEND }}
```

5. github action을 이용한 vercel 배포로 전환

❖백엔드 배포를 위한 workflow 파일 예시

- backend-prod.yaml : test 과정 없음

```
name: Backend Production
on:
  push:
    branches: [main]
    paths:
      - "backend/**"
jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.VERCEL_ORG_ID }
          vercel-project-id: ${ secrets.VERCEL_PROJECT_ID_BACKEND }
          vercel-args: "--prod"
```

5. github action을 이용한 vercel 배포로 전환

❖프론트엔드 배포를 위한 workflow 파일 예시

▪ frontend-preview.yaml

```
name: Frontend Preview

on:
  push:
    branches: [preview]
    paths:
      - "frontend/**"
  pull_request:
    branches: [main]
    paths:
      - "frontend/**"

jobs:
  test-and-build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [22.x]
```

```
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js ${{ matrix.node-version }}
    uses: actions/setup-node@v4
    with:
      node-version: ${{ matrix.node-version }}
      cache: "npm"
      cache-dependency-path: "./frontend/package-lock.json"

  - name: Install dependencies
    working-directory: ./frontend
    run: npm ci

  - name: Run tests
    working-directory: ./frontend
    run: npm run test
```

5. github action을 이용한 vercel 배포로 전환

❖ 프론트엔드 배포를 위한 workflow 파일 예시

▪ frontend-preview.yaml(이어서)

```
deploy:
  needs: test-and-build
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Deploy to Vercel
      uses: amondnet/vercel-action@v25
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }
        vercel-org-id: ${ secrets.VERCEL_ORG_ID }
        vercel-project-id: ${ secrets.VERCEL_PROJECT_ID_FRONTEND }
```

5. github action을 이용한 vercel 배포로 전환

❖프론트엔드 배포를 위한 workflow 파일 예시

▪ frontend-prod.yaml

```
name: Frontend Production
on:
  push:
    branches: [main]
    paths:
      - "frontend/**"
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${ secrets.VERCEL_PROJECT_ID_FRONTEND }}
          vercel-args: "--prod"
```


5. github action을 이용한 vercel 배포로 전환

❖프론트엔드 배포 예시

Summary

Jobs

test-and-build (22.x)

deploy

Run details

Usage

Workflow file

Triggered via push yesterday

Status

Total duration

Artifacts

stepanowon pushed

061d200

main

Success

1m 30s

-

frontend-preview.yaml

on: push

Matrix: test-and-build

1 job completed

Show all jobs

deploy58s

-

+

6. 다른 환경에 배포하기 위한 github action 예시

❖ Container 이미지를 빌드하고 Docker Hub에 푸시하는 workflow 예시

```
name: Docker Hub Sample
on:
  push:
    branches: [main, develop]
    tags:
      - 'v*'
jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }
```

```
- name: Extract metadata
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: your-username/your-image-name
    tags: |
      type=ref,event=branch
      type=ref,event=pr
      type=semver,pattern={{version}}
      type=semver,pattern={{major}}.{{minor}}.{{build}}
      type=sha

- name: Build and push
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ${ steps.meta.outputs.tags }
    labels: ${ steps.meta.outputs.labels }
    cache-from: type=registry,ref=your-username/your-image-
name:builddcache
    cache-to: type=registry,ref=your-username/your-image-
name:builddcache,mode=max
```

6. 다른 환경에 배포하기 위한 github action 예시

❖ Spring Boot 앱을 빌드하고 SSH로 배포하는 Workflow 예시

```
name: Build and Deploy Spring Boot (Maven)
on:
  push:
    branches: [main]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'
      - name: Build with Maven
        run: mvn clean package -DskipTests
      - name: Run tests
        run: mvn test
```

```
- name: Deploy to Server
  uses: appleboy/ssh-action@v1.0.3
  with:
    host: ${ secrets.SSH_HOST }
    username: ${ secrets.SSH_USERNAME }
    key: ${ secrets.SSH_PRIVATE_KEY }
    port: ${ secrets.SSH_PORT }
    script: |
      cd /home/app
      sudo systemctl stop myapp

- name: Copy JAR to Server
  uses: appleboy/scp-action@v0.1.7
  with:
    host: ${ secrets.SSH_HOST }
    username: ${ secrets.SSH_USERNAME }
    key: ${ secrets.SSH_PRIVATE_KEY }
    port: ${ secrets.SSH_PORT }
    source: "target/*.jar"
    target: "/home/app"
    strip_components: 1
```

6. 다른 환경에 배포하기 위한 github action 예시

❖ Spring Boot 앱을 빌드하고 SSH로 배포하는 Workflow 예시(이어서)

```
- name: Start Application
  uses: appleboy/ssh-action@v1.0.3
  with:
    host: ${ secrets.SSH_HOST }
    username: ${ secrets.SSH_USERNAME }
    key: ${ secrets.SSH_PRIVATE_KEY }
    port: ${ secrets.SSH_PORT }
    script: |
      cd /home/app
      mv *.jar myapp.jar
      sudo systemctl start myapp
      sleep 5
      curl -f http://localhost:8080/actuator/health || exit 1
```

7. 정리

❖ 운영 데이터베이스 준비

- MCP를 활용하여 로컬 데이터베이스 스키마와 동일하게 Supabase 데이터베이스 준비
- DB Connection String 확인

❖ Vercel for github을 이용한 배포 테스트

- 환경변수 지정, 작동 여부 확인

❖ Github Action을 이용한 CI/CD

- Github Action 구성 요소 : Workflow, Event, Job, Step, Action
- Github Action을 이용한 Vercel 배포 테스트
- 다른 배포환경에 대한 설정 방법